UVM Interview Questions

Q1: What is UVM? What is the advantage of UVM?

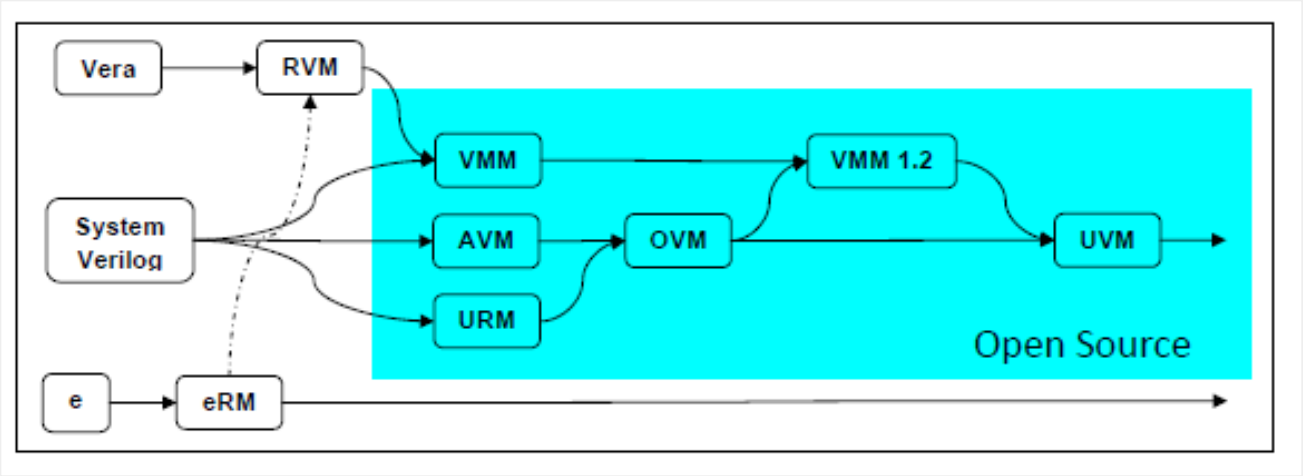
Ans: UVM (Universal Verification Methodology) is a standardized methodology for verifying the both complex & simple digital design in simple way.

UVM Features:

- First methodology & second collection of class libraries for Automation
- Reusability through testbench
- Plug & Play of verification IPs
- Generic Testbench Development
- Vendor & Simulator Independent
- Smart Testbench i.e. generate legal stimulus as from pre-planned coverage plan
- Support CDV –Coverage Driven Verification
- Support CRV –Constraint Random Verification
- UVM standardized under the Accellera System Initiative
- Register modeling

Q2: UVM derived from which language?

Ans: Here is the detailed connection between SV, UVM, OVM and other methodologies.



Q3. What is the difference between uvm_component and uvm_object?

OR

We already have uvm_object, why do we need uvm_component which is actually derived class of uvm_object?

Ans:
uvm_component:

- Quasi Static Entity (after build phase it is available throughout the simulation)
- Always tied to a given hardware(DUT Interface) Or a TLM port
- Having phasing mechanism for control the behavior of simulation
- Configuration Component Topology

uvm_object:

- Dynamic Entity (create when needed, transfer from one component to other & then dereference)
- Not tied to a given hardware or any TLM port
- Not phasing mechanism

Q4: Why phasing is used? What are the different phases in uvm?

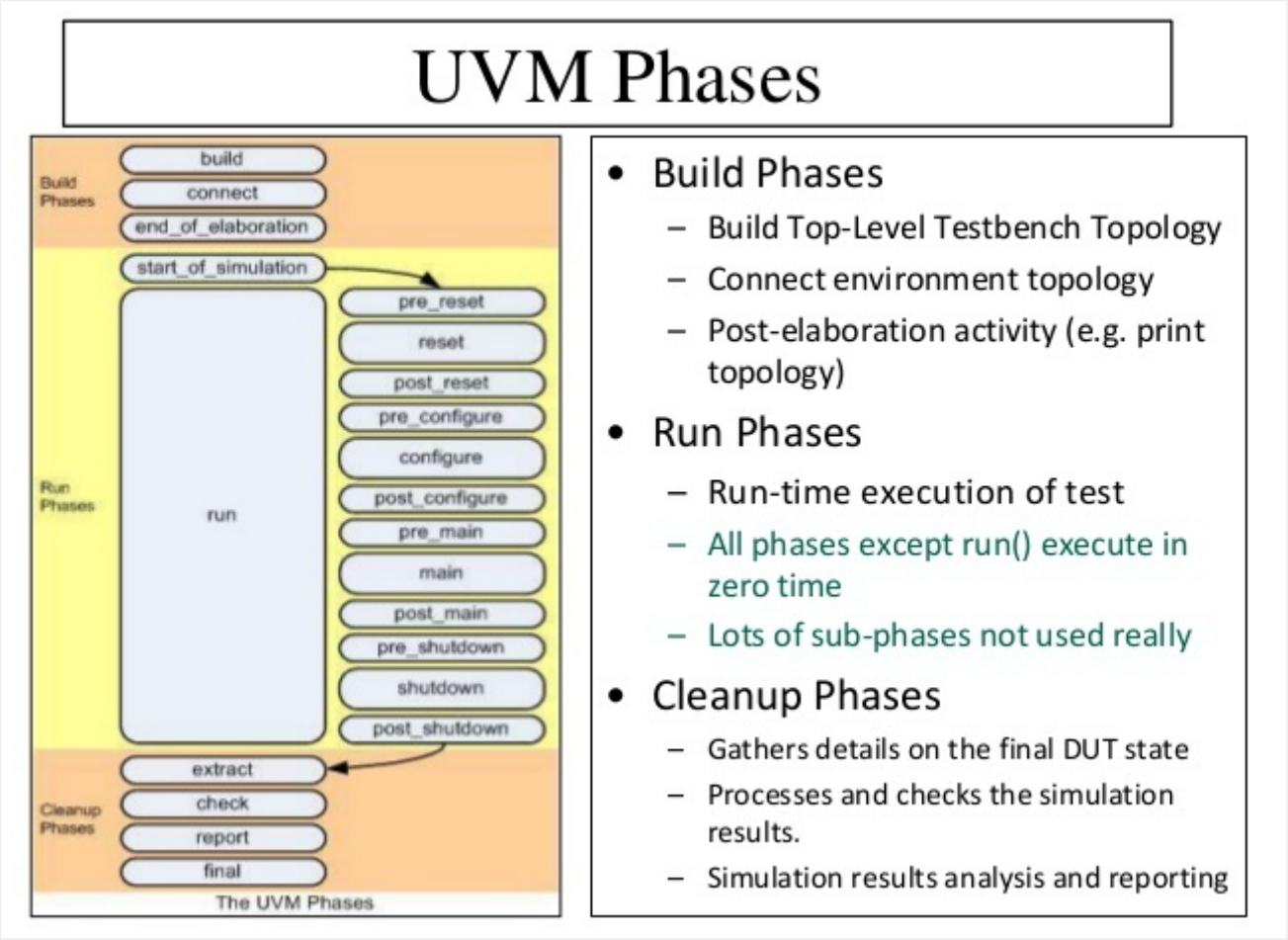
Ans: UVM Phases is used to control the behavior of simulation in a systematic way & execute in a sequential ordered to avoid race condition. This could also be done in system verilog but manually.

1. List of UVM Phases:
2. buid_phase
3. connect_phase
4. end_of_elaboration_phase
5. start_of_simulation_phase
6. run _phase (task)
   Sub Phases of Reset Phase:
   pre_reset_phase
   reset_phase
   post_reset_phase
   pre_configure_phase
   configure_phase
   post_configure_phase
   pre_main_phase
   main_phase

post_main_phase
pre_shutdown_phase
shutdown_phase
post_shutdown_phase

7. extract_phase
8. check_phase
9. report_phase

Below figure makes it more clear



Q5: Which uvm phase is top - down , bottom – up & parallel?

Ans: Only build phase is a top-down & other phases are bottom-up except run phase which is parallel. The build phase works top-down since the testbench hierarchy may be configure so we need to build the branches before leafs

Q6: Why build phase is top – down & connect phase is bottom – up?

Ans: The connect phase is intended to be used for making TLM connections between components, which is why it occur after build phase. It work bottom-up so that its got the correct implementation all the way up the design hierarchy, if worked top-down this would be not possible

Q7: Which phase is function & which phase is task?

Ans: Only run phase is a task (time consuming phase) & other phases are functions (non-blocking)

Q8: Which phase takes more time and why?

Ans: As previously said the run phase is implemented as task and remaining all are function. run phase will get executed from start of simulation to till the end of simulation. run phase is time consuming, where the testcase is running.

Q9: How uvm phases initiate?

Ans: UVM phases initiate by calling run_test("test1") in top module. When run_test() method call, it first create the object of test top & then call all phases.

Q10: How test cases run from simulation command line?

Ans: In top module write run_test(); i.e. Don't give anything in argument.
Then in command line : +UVM_TESTNAME=testname